Name: | Email: | @innopolis.university

INNOPOLIS UNIVERSITY



Dear student, welcome to the Admission Test!

Please, read the following information about the test:

- This is an **open book** test:
  1. Notes, cheat sheets, and devices (laptops, tablets, phones, smartwatches) are **allowed**.
  2. However, communicating with other students during the test will result in immediate removal from the admission test, without any opportunity of retake.
- **Do not** open the questions before the test starts!
  Reading questions before the official start will be considered cheating.
- Keep your student ID card on the table next to you for the duration of the test.
- Write your name and email address at the top this page.
- When the test starts, write your name and email address at the top of the second sheet of paper.
- This test contains 20 questions for a total of 20 points (all questions weigh the same).
- You must score **at least 10 points** to PASS the admission test.
- While you wait for the test to start, you may draw a doodle in the box below or read this:

Name: _____ Email: _____@innopolis.university

1. Select **all** λ-terms that reduce to $\lambda x.\lambda y.y$ (or to an α-equivalent term):

   ☐ $(\lambda x.\lambda y.\lambda z.z)\ (\lambda x.y)$

   ☐ $(\lambda x.\lambda y.\lambda z.x)\ z$

   ☐ $(\lambda x.\lambda y.x\ y)\ (\lambda z.\lambda w.w)$

   ☐ $(\lambda x.x\ x)\ (\lambda y.\lambda z.\lambda w.w)$

2. Select **all** evaluation strategies that **do not** evaluate subterms under λ-abstraction:

   ☐ full β-reduction

   ☐ applicative order

   ☐ normal order

   ☐ call-by-name

   ☐ call-by-value

   ☐ none of the above

3. It is possible to show that `(map (lambda (x) (f (g x))) lst)` is equivalent to `(map f (map g lst))` for all `f`, `g` and `lst` using

   ☐ DrRacket

   ☐ α-equivalence

   ☐ β-reduction

   ☐ accumulator parameter pattern

   ☐ tail recursion

   ☐ substitution model

   ☐ Racket interpreter

   ☐ none of the above

4. Which of the following are Racket functions (not special forms)?

   ☐ `map` ☐ `reverse`

   ☐ `lambda` ☐ `cond`

   ☐ `or` ☐ `ormap`

   ☐ `+` ☐ `else`

   ☐ `define` ☐ `length`

5. Select **all** true statements:

   ☐ Haskell allows working naturally with (potentially) infinite data structures.

   ☐ In miniKanren, negation is usually done using Racket's `not` function.

   ☐ Racket is dynamically typed.

   ☐ In Haskell, it is considered bad practice to rely on partial functions.

   ☐ The logic engine of miniKanren is implemented using depth-first search.

   ☐ In Haskell, type signatures are optional, because it is dynamically typed.

   ☐ Higher-order functions and algebraic data types are the most important concepts of functional programming.

   ☐ In logic programming, a key feature is the ability to execute predicates/relations in different modes.

6. Select **all** statically typed languages that support exhaustive pattern matching:

   ☐ Java (latest) ☐ C++

   ☐ Racket ☐ Swift

   ☐ Haskell ☐ Rust

   ☐ Python ☐ JavaScript

   ☐ Scala ☐ miniKanren

7. Select **all** Haskell expressions which will result in a **type error**?

   ☐ `[("λ", 1)]` ☐ `floor (1 + 2.5)`

   ☐ `floor 1 + 2.5` ☐ `map (+1) [1..]`

   ☐ `map (+)` ☐ `let x = x : [] in x`

   ☐ `["λ", 1]` ☐ `let x = [] : x in x`

8. Select **all** Haskell expressions that will evaluate in finite time (i.e. not hang):

   ☐ `sum (take 10 (reverse [1..]))`

   ☐ `sum (take 10 ([1..] ++ [1..]))`

   ☐ `take 1 (reverse (reverse [1..]))`

   ☐ `find (<0) [1..]`

   ☐ `take 1 (filter (>10) [1..])`

9. Write down the (most general) type for each of these Haskell expressions:

   `Just Nothing ::` _____

   `filter (> 0) ::` _____

   `map reverse ::` _____

   `map (map Just) ::` _____

   `\x -> x:[x] ::` _____

10. For each polymorphic Haskell type, write down the number of distinct **total** function implementations with that type:

    `[a] -> a` _____

    `Maybe a -> a` _____

    `Maybe a -> Maybe a` _____

    `a -> Maybe a` _____

    `(a, a) -> Maybe a` _____

    `(a -> b) -> a -> [b]` _____

Name: _____ Email: _____ @innopolis.university

11. Which of the following Racket special forms always produce a list?

- ☐ `quote`
- ☐ `for*`
- ☐ `for/list`
- ☐ `for/or`
- ☐ `for/fold`
- ☐ `for`
- ☐ `for/and`
- ☐ `for*/list`

12. Which key primitives does μKanren (microKanren) implement?

- ☐ `absento`
- ☐ `run`
- ☐ `call/fresh`
- ☐ `conj`
- ☐ `numbero`
- ☐ `=/=`
- ☐ `matche`
- ☐ `==`
- ☐ `build-num`
- ☐ `disj`
- ☐ `symbolo`
- ☐ `not`

13. Select the **minimal** number of primitives that are needed to define `not-member`^{o} relation. `(not-member`^{o} `x xs)` succeeds when `x` is not an element of `xs`.

- ☐ `absento`
- ☐ `run`
- ☐ `fresh`
- ☐ `numbero`
- ☐ `=/=`
- ☐ `matche`
- ☐ `==`
- ☐ `conde`
- ☐ `symbolo`
- ☐ `not`

14. How many times will the following Haskell program output `"hello"`?

```
main :: IO ()
main = do
  putStrLn "hello"
  x <- return ["hello", "hello"]
  return x
  mapM_ print x
  y <- return (putStrLn "hello")
  y
  return y
```

Answer: _____

15. Write down the types of these standard higher-order Haskell functions:

`takeWhile` :: _____

`map` :: _____

`filter` :: _____

`find` :: _____

`zipWith` :: _____

16. Which of the following Haskell functions are total?

- ☐ `concat`
- ☐ `head`
- ☐ `find`
- ☐ `tail`
- ☐ `(!!)`
- ☐ `read`
- ☐ `putStrLn`
- ☐ `reverse`

17. Assuming local variables `n :: Int` and `x :: Double`, fill in the gaps with proper Haskell type conversion functions:

`x -` _____ `(n +` _____ `x)`

18. Write down the name of instructors in this course:

Lecturer _____

Your TA _____

19. Write down the room number that you are currently in:

Room _____

20. Select the homework assignment that you enjoyed doing the most:

- ☐ Homework 1 (in Racket)
- ☐ Homework 2 (in Haskell)
- ☐ Homework 3 (in miniKanren)

Have you double-checked your answers? Do you want to leave a comment?

Name: | Email: | @innopolis.university

You may use this page as a draft.